

Lab # 11

Shell Control Structures

I. The if Statement

Remember that the spacing is very important in the if statement. Notice that the termination of the if statement is `fi`. You will need to use the `fi` statement to terminate an if whether or not use an else as well. You can also replace the `"=="` with `"!="` to test if the variables are NOT equal. There are other tokens that you can put in place of the `"=="` for other types of tests. The following table shows the different expressions allowed.

Comparisons:

-eq	equal to
-ne	not equal to
-lt	less than
-le	less than or equal to
-gt	greater than
-ge	greater than or equal to

a) The if Statement

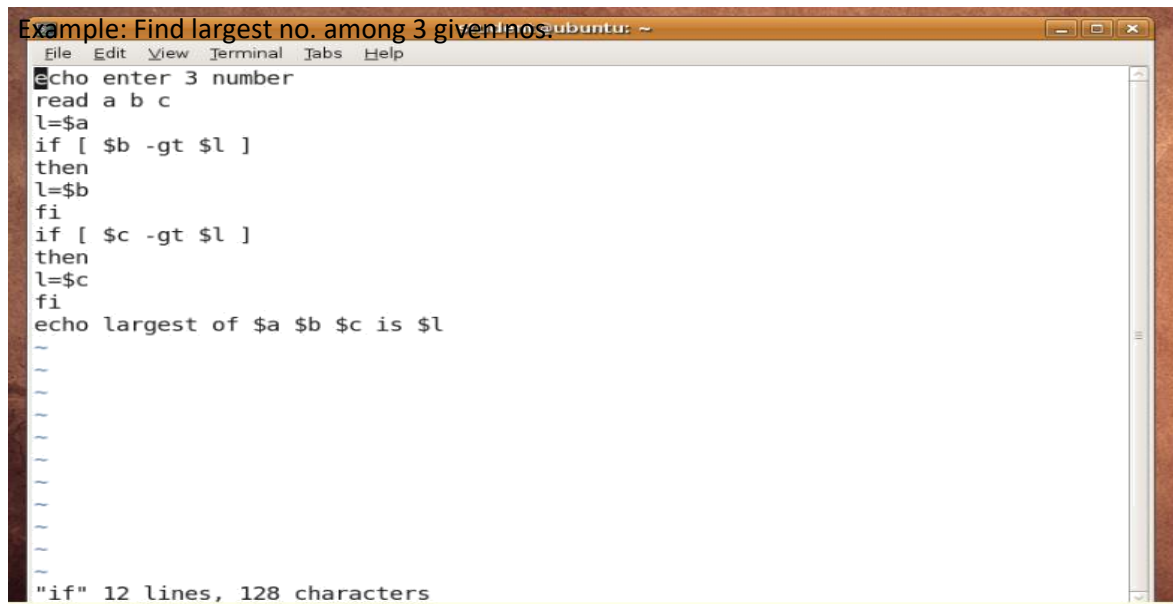
Example: Sum two integers

```
$ vi suminteger.sh
```

```
# To find sum of two integral values
if [ $# -ne 2 ] then      echo "Please input two integer values"
exit 1      fi
echo "Sum of $1 & $2 is `expr $1+$2`"
```

```
$ ./suminteger.s
```

b) Multilevel if... then... elif... else... fi



The screenshot shows a terminal window titled "ubuntu: ~" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The script content is as follows:

```
echo enter 3 number
read a b c
l=$a
if [ $b -gt $l ]
then
l=$b
fi
if [ $c -gt $l ]
then
l=$c
fi
echo largest of $a $b $c is $l
```

Below the script, there are several tilde (~) characters representing input. At the bottom, a status bar indicates: "if" 12 lines, 128 characters.

\$vi largest

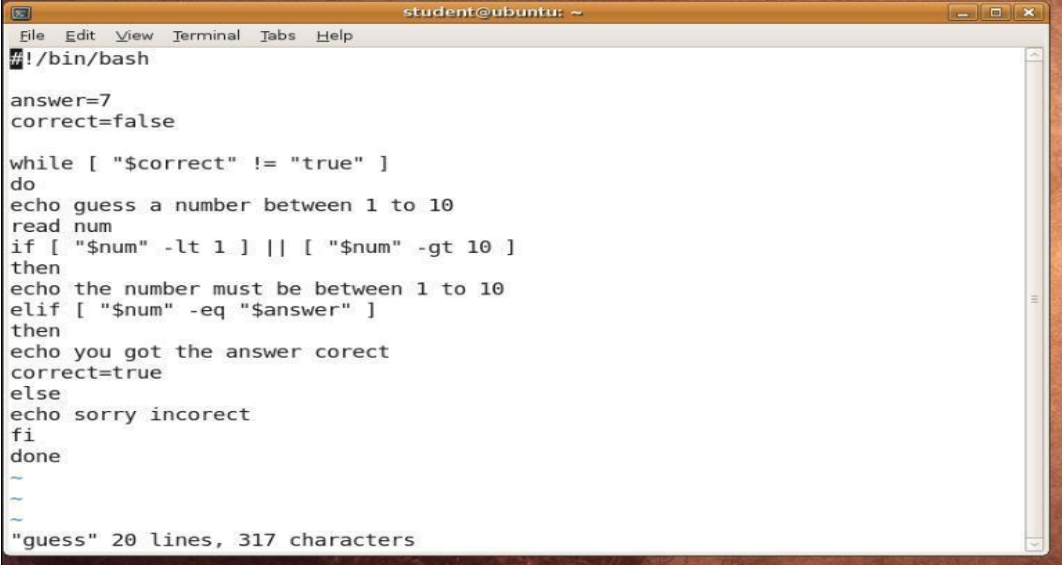
```
#largest no.
If [$# -ne 3]
then
echo "Please give 3 nos."
read n1
read n2
read n3
fi
n1 = $1
n2 = $2
n3 = $3
if [$n1 -gt $n2]
then
    if [$n1 -gt $n3]
    then
        echo "$n1 is largest"
    else
        echo "$n3 is largest"
    fi
elif [$n2 -gt $n3]
then
    echo "$n2 is largest"
else
    echo "$n3 is largest"
fi
fi
```

II. while... do... done

Example: Display 1 to 10 nos.

#while loop

```
count=1 while [ $count -le 10 ] do echo $count count = `expr $count + 1`  
done
```



```
student@ubuntu: ~  
File Edit View Terminal Tabs Help  
#!/bin/bash  
  
answer=7  
correct=false  
  
while [ "$correct" != "true" ]  
do  
    echo guess a number between 1 to 10  
    read num  
    if [ "$num" -lt 1 ] || [ "$num" -gt 10 ]  
    then  
        echo the number must be between 1 to 10  
    elif [ "$num" -eq "$answer" ]  
    then  
        echo you got the answer correct  
        correct=true  
    else  
        echo sorry incorrect  
    fi  
done  
  
~  
~  
~  
"guess" 20 lines, 317 characters
```

```
#      !/ b i n / b a s h # Guess the number game.
```

```
ANSWER= 5 # The correct answer
```

```
CORRECT=false # The correct flag
```

```
while [ "$ CORRECT" != "true" ] do
```

```
# Ask the user for the number...
```

```
echo "Guess a number between 1 and 10 . " read NUM
```

```
# Validate the input ...
```

```
$/while.sh
```

```

if [ "$ NUM" -lt 1 ] || [ "$ NUM" - gt 10 ];
then
echo "The number must be between 1 and 10!"
elif [ "$ NUM" -eq "$ ANSWER" ];
then
echo "You got the answer correct!"
    CORRECT=true
else
echo "Sorry, incorrect . "
fi
done

```

III. for... do... done

```

fori in 1 2 3 4 5 do

```

```

{ code} done

```

The **for** statement is used when you want to loop through a list of items . The body of the loop is put between **do** and **done**. Let's say that we want to write a program that will validate numbers in a given list. These numbers can be loaded from a file, hard coded, or manually entered by the user. For our example, we will ask the user for a list of numbers separated with spaces. We will validate each number and make sure that it is between 1 and 100. The best way to write a program like this would be to use a **for** loop.

```

#!/bin/sh
# Validate numbers...

echo "Please enter a list of numbers between 1 and 100 . " read NUMBERS

for NUM in $ NUMBERS do
if [ "$ NUM" -lt 1 ] || [ "$ NUM" - gt 100 ]; then
    echo "Invalid Number ($ NUM) - Must be between 1 and 100!"
else echo "$ NUM is valid . "
    fi
done

```

```

Please enter a list of numbers between 1 and 100 .
1435548120100041
1 is valid .
4 is valid . 3 is valid .
55 is valid . 48 is valid .

```

```

InvalidNumber (120 )-Must be between 1 and 100 !
InvalidNumber (1000 ) -Must be between 1 and 100 !
4 is valid .
1 is valid .

```

a) Nested Loop

Example: All combination of nos. 1,2& 3

```
a=1
while [ $a -le 3]
do
    b=1
    while [$b -le 3]
    do
        c=1
        while [$c -le 3]
        do
            echo "$a$b$c"
            c=`expr $c+1`
        done
        b=`expr $b+1`
    done
    a=`expr $a+1`
done
$./nested.sh
```

Lab Tasks

1. Write a program to generate the following output by using while loop

The value of x is 10
The value of x is 9
The value of x is 8
The value of x is 7
The value of x is 6
The value of x is 5
The value of x is 4
The value of x is 3
The value of x is 2
The value of x is 1

2. Write a program to generate the table of any number by using for loop.
3. Write a program to generate the factorial of a number.
4. Write a program to show the grades and % of students (with students details).